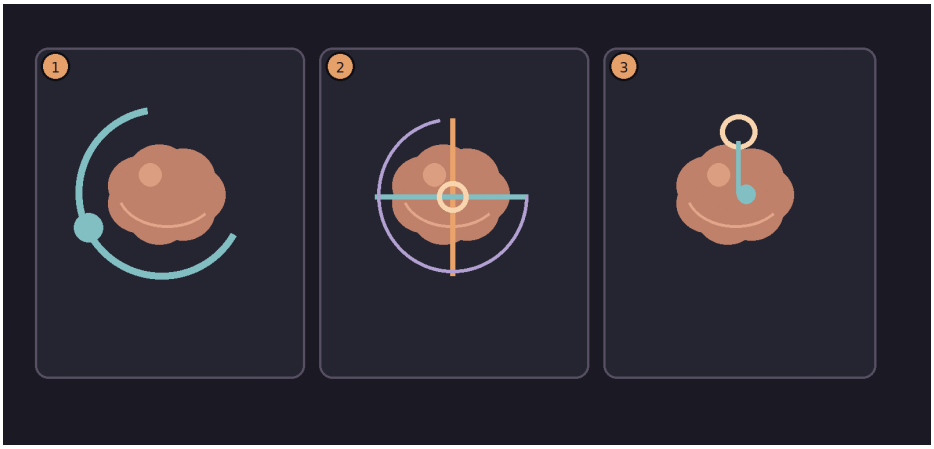
BUILD WORLDS TOGETHER

A four-product interface comparison and a Quest-native multiplayer creation architecture

Substance 3D Modeler | Gravity Sketch | Horizon Worlds | Multibrush



Original technical illustration from the previous reference. Not a product screenshot.

> **Decision in one sentence.** A standalone Quest creator app is feasible. Build it as a layered scene editor with fast spatial strokes, editable parametric surfaces, limited local SDF sculpting, and synchronized object commands. Do not attempt to reproduce the complete desktop Substance engine in a headset.

Coverage: Desktop and VR controllers, interface regions, feature inventories, scene representations, collaboration models, Quest rendering and memory constraints, data structures, networking, object ownership, export, accessible UX, phased engineering delivery, and an actual-screenshot atlas.

Scope and accuracy: Source facts and engineering proposals are separated. Screenshots are linked to the publisher or captured documentation; accompanying vector diagrams are original analysis. Some historic Horizon and Multibrush screenshots depict earlier UI. Sources reviewed in October 2026.

01 | What each application contributes

Application	Core interaction	Best reusable concept	Limit for a Quest-native collaboration app
Substance 3D Modeler	Clay SDFs, non-destructive procedural primitives, brush deformation	Sculpting UX, support/tool hands, layer focus, expressive clay	Windows PC only; memory and compute demand; no native shared live-sculpt protocol documented
Gravity Sketch	6DoF strokes, surfaces, curves, SubD topology, precision control	Editable structured geometry, collaborative rooms, spatial scale	Precision/geometry complexity; some Quest-only feature limitations
Horizon Worlds	Scene assembly, prefabs, interaction scripts, social world runtime	Creator/visitor separation, runtime logic, avatar/world semantics	Not a full surface-modeling engine; 2026 Worlds roadmap mobile-first
Multibrush	Room-scale painterly strokes and shared painting	Fast low-latency co-creation, inviting tool palette	Some tools missing in multiplayer rooms; not a parametric or voxel sculpt system

Do not conflate modalities. Drawing a tube in midair, editing a spline control point, deforming a signed-distance volume, and moving a physics-enabled world object are four different operations. They should appear coherent to creators but not be stored or networked identically.

Where the capabilities overlap

Capability	Modeler	Gravity Sketch	Horizon Worlds	Multibrush
Quest standalone creation	No	Yes	Historical VR build, subject to 2026 policy	Yes
Native PC desktop editor	Yes	Desktop primarily review/collab	Yes	No equivalent full desktop editor
Live multiuser creation	Not documented as native	Yes	Historically yes	Yes, selected tools
Volumetric clay SDF	Yes	No equivalent editor	No	No
Editable surfaces/SubD	Not primary workflow	Yes	Imported meshes/prefabs	No equivalent editor
Painterly spatial strokes	Sculptural brush	Yes	Not core	Yes
In-world runtime logic	Not core	Not core	Yes, TypeScript	Not core
MR passthrough	VR configuration dependent	Yes (standalone)	Platform-dependent experiences	Yes
Ownable export-oriented geometry	Meshes and scene formats	Yes	Imported assets/world publishing model	Brush artwork export varies

This matrix distinguishes documented product strengths, not feature parity. A generic word such as "paint" can mean vertex/voxel color in Modeler, color or material on a designed surface in Gravity Sketch, mesh tint or asset behavior in Worlds, and rendered stroke geometry in Multibrush.

02 | Why Adobe requires a computer, and why Quest can do more today

Adobe officially says Modeler does not execute standalone on a headset. It runs on Windows and streams VR output through a supported PC connection, wired or high-speed wireless. Its current minimum specifications list an 8 GB dedicated GPU, 16 GB system memory, and an SSD. Meta Quest 3 can supply tracking and stereo display, but the PC remains the compute host. [S1, S2]

Product architecture is a choice, not an impossibility theorem

- **Compute:** SDF editing requires evaluating field samples, remeshing affected regions, and updating GPU buffers. High resolution, large brushes, and dense layers have nontrivial CPU/GPU workloads.

- **Memory:** An illustrative dense 256-cube grid holds 16,777,216 samples. Just one 32-bit distance plus one 32-bit color value is about 134 MB before acceleration structures, multiple clay layers, undo, meshes, shading, and system memory. At 512 cubed this pair alone approaches 1.07 GB.

- **Rendering:** Two-eye low-latency rendering plus pass-through, hands, UI, avatars, and networked world geometry compete for mobile resources. A creator app must remain responsive even while a geometry update is in progress.
- **Engineering/history:** Modeler was built as a desktop-class authoring system using OpenXR as its VR interface. An optimized Quest-native renderer, Android packaging, file/cloud workflows, and mobile memory strategy would be a separate engineering undertaking.
- **Not proof of intrinsic limitation:** Multibrush and Gravity Sketch operate standalone on Quest because their scene representations, feature budgets, and rendering paths were engineered to suit it. A new app can make similar choices without duplicating every Modeler feature.

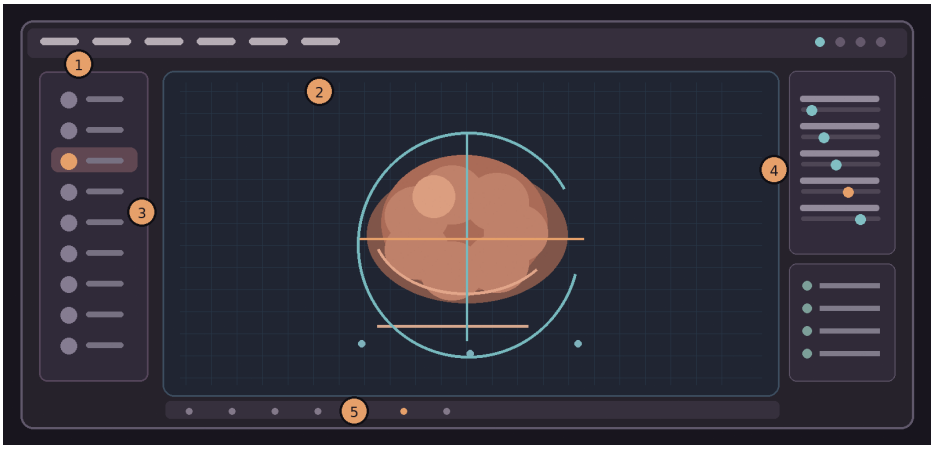
What has changed, 2016 to 2026

Period	Technical shift	Practical lesson
2016-2021	Consumer tracked controllers enable spatial painting. Tilt Brush source becomes available in 2021; MultiBrush rapidly builds multiplayer drawing.	A 3D creative action can be transmitted as lightweight stroke data.
2022-2024	Standalone Quest workflows mature: Multibrush avatars and passthrough; Gravity Sketch full VR creation, collaborative rooms.	Mobile GPUs can sustain rich authoring when geometry and shaders are constrained.
2025	Gravity Sketch improves tool presets, online rooms, lighting and collaborative screen review. Worlds desktop editor supports mesh imports and TypeScript.	Editors, viewers and co-creators can use different devices in the same workflow.
2026	Meta documentation covers shared anchors, colocation sessions and Fusion-based multiplayer samples; Horizon Worlds directs new work toward mobile.	Build a native VR creator product on Meta XR APIs rather than depending on the Worlds publishing ecosystem.

Performance arithmetic. A 72 Hz frame lasts about 13.9 milliseconds. That is a complete-frame deadline, not a 13.9 ms budget for sculpting alone. Meta requires interactive Quest applications to achieve at least 72 frames per second. Our proposed smaller engineering budgets below are test targets, not platform mandates. [S3]

03 | Substance 3D Modeler: complete operating model

Desktop workspace



Original labeled UI reconstruction, not an Adobe screenshot. See the accompanying screenshot atlas for the actual Modeler windows.

Part	What it controls	Desktop behavior
Top bar	Menus, workspace/mode, panels, preferences	Mouse to open menus or panels; keyboard shortcuts where supported
Viewport	Sculpt and selection workspace, 3D camera	Alt plus mouse: orbit/pan/zoom; sculpt with mouse or pen
Palette	Active tool, subtools, brush properties, shapes	Click tools; edit numeric parameters or sliders
Actions menu	Contextual creation, grouping, duplication, visibility, resolution	Right-click in viewport; actions depend on scope or selection
Outliner/hierarchy	Layer and group relationships	Select, rename, hide, group; manage parent and child structure
Bottom bar	Placement/context modes and status	Toggle brush placement behavior and inspect context

Official clay/sculpt tool inventory

Tool	Effect and editing semantics	Typical use
Select	Targets layers and scene objects; transform with gizmo; context actions	Organize, move, duplicate, delete
Clay	Adds volume from empty space into the active clay layer	Block forms and shapes
Erase	Removes volume where the brush intersects in-scope clay	Cut grooves, remove material
Split	Separates material into other clay layers	Segment a sculpt for independent edits
Crop	Keeps intersection and removes external volume	Trim to profile or brush boundary
Warp	Moves/extends/twists existing clay with six-degree motion	Pull features, make tendrils
Elastic	Warp-like shape change favoring retained volume	Refine without strong volume loss
Buildup	Raises/lowers existing surface	Subtle form layering
Crease	Carves or raises narrow fold/ridge	Seams and sculptural edges
Smooth	Smooths uneven regions of a surface	Blend lumpy brush strokes
Flatten	Pushes surfaces toward an inferred or locked plane	Planar surfaces, bevel preparation
Inflate	Pushes local surfaces outward/inward	Swell or shrink volume
Paint	Changes color stored with clay, interacting with material settings	Color and visual variation

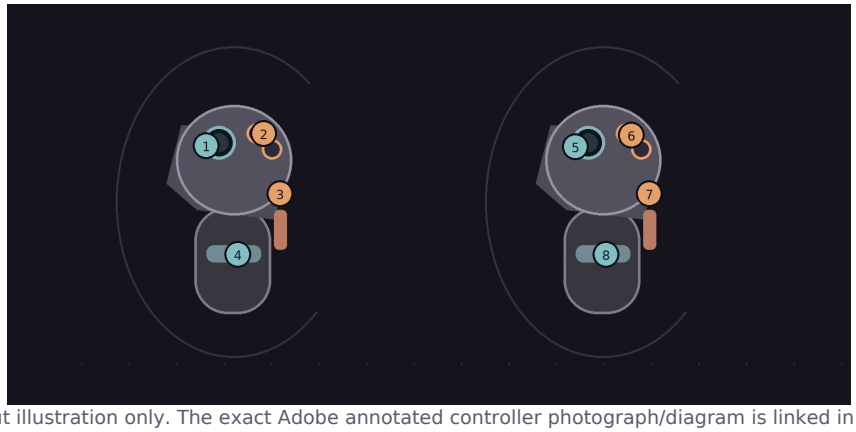
Non-destructive primitives: a different engine inside Modeler

Modeler also supports parametric SDF primitives assembled in a container. Shape functions, combine modes (including carving/subtraction), blending, ordering, grouping, and repetition can produce complex forms without destructively painting voxels. Converting primitives to clay enables brush editing, but is not automatically reversible except by undo. This makes a good model for a Quest app: retain parametric descriptions while possible; rasterize locally and only when a tool genuinely needs clay. [S4]

Concrete desktop shortcuts

Input	Action	Note
C / E / W / B	Clay / Erase / Warp / Buildup	Direct creation tools
V / X / D	Select / Crease / Flatten	Official control table baseline
Tab	Switch to tool companion	E.g. Clay and Erase
Hold Shift	Temporary Smooth brush	Sculpting context
Ctrl + mouse drag	Brush size	Modeler desktop
Alt + left/middle/right drag	Orbit / pan / zoom	Viewport gestures
G	Toggle brush placement	Gizmo vs free placement (check version)
Right-click	Open contextual Actions	Scope-sensitive
F4	Manually enter/leave VR mode	When automatic handover is not available

Actual VR controller mapping, handedness-neutral



Original simplified layout illustration only. The exact Adobe annotated controller photograph/diagram is linked in the screenshot atlas.

Physical input	Official function	Design implication
Support button S2	Show Palette; while held use tool stick to switch Palette and Actions	Temporary hand-anchored UI
Support button S1	Toggle gizmo placement	Fast precision-mode switch
Support stick left/right	Undo / redo	Undo must be creator-scoped online
Support stick up/down	Scope into / out of objects and groups	Hierarchical scene focus in VR
Support stick click	Color-pick mode	Can avoid opening a panel
Support trigger ST	Tool modifier; duplicate while moving, move brush origin	Bimanual alternate behavior
Support grip SG	Move world; together with tool grip, scale world	World manipulation, seated accessibility

Physical input	Official function	Design implication
Tool trigger TT	Use active tool, pick menu items	Pressure-based brush possible
Tool button T1	Quicktune, context-specific selection	Use thumbstick/gesture while held
Tool button T2	Tool quick swap; select-context delete	High-frequency shortcut
Tool stick up/down	Brush size	Continuous tactile adjustment
Tool stick left/right	Brush hardness where applicable	Keeps eyes on sculpt
Tool grip TG	Move world; dual-grip scale	Symmetric scene navigation

Adobe maps controls to **support hand** and **tool hand**, rather than absolute left and right. For right-handed mode, tool hand is right; handedness can be mirrored in settings. One grip moves the whole scene; gripping with both hands and changing hand separation scales the whole scene. In VR, the support-hand Palette is pointed at and operated with the tool trigger. [S5]

> **Documented inconsistency.** Some older Adobe help passages call the Palette button S1 while the dedicated reference states S2. This report follows the dedicated VR shortcut table, flags version differences, and recommends reading the in-app button tooltip on a live build.

04 | Gravity Sketch: spatial precision and editable surfaces

Its user interaction grammar

Controller/input	Documented behavior	Why it works
Dominant index trigger	Sketch, confirm tool, point to menu selection	Draws directly in space
Dominant grip	Grab objects, move/rotate/scale objects	Manipulation without a transform dialog
Dominant thumbstick	Resize grab sphere or stroke; modify edit selection	Precision and reach adjustment
Dominant red button	Undo when free; delete while grabbing	One high-frequency context key
Non-dominant blue button	General Menu, edit settings and file operations	Persistent system actions on other hand
Non-dominant purple button	Creation tools, four tool categories and brush presets	Gesture-first tool choosing
Non-dominant grip	Move/rotate the environment; with both grips scale self relative to work	Scale navigation and seated work
Non-dominant thumbstick	Navigate edit/history controls by context	History without opening a menu
Half-trigger	Axis or plane snap in supported contexts	Precision in gestural UX
Wrist turn in/out	Quick Access menus for exit, screenshots and navigation	Low-clutter transient UI
Color wheel	Change color and material	Immediate visual feedback

Enumerated creator tools and editable geometry

Group	Feature inventory	Data representation / effect
Draw	Ink strokes, stroke/tube drawings, volumetric drawing; reusable brush presets	Sampled control points with radius, color, material
Surface	Two-hand ribbon/surface drawing; revolve from curves, surfacing controls	Editable surface patches and profiles
Shapes	11 documented primitive shape choices	Parameterized primitives and modeled surfaces
Text	Three-dimensional text object, font/type controls via keyboard	Text as object/mesh, not a raster drawing
Surface editing	Subdivision/SubD, extrude, thicken/offset, roundedness, edge cut, insert loops, crease, merge/separate	Topology edit on editable control cages
Selection/edit	Select, grab, move, group/ungroup, duplicate, soft select, edit curve handles	Object and control-point updates
Guides	Mirror, snapping, axes, grid, measurements, 1:1 scale, Smart Move	Constraints/coordinate systems
Visuals	Color/material, transparency, lighting/environments, shadows, reference images and model imports	Render and reference layers
Scene/communication	Layers, screenshots, viewpoints, notes, marks, voice/chat tools in supported collaboration clients	Shared session metadata and annotations
Experimental	Labs image-to-3D, one-controller scaling, passthrough depth occlusion (hardware-specific)	May change, disappear or be experimental

Gravity Sketch documents **four categories of sketch tools, with six customizable named presets per category**. It also supports real-time collaborative rooms, invitations, room codes and web/Desktop Screen Collab reviewing. VR is the full authoring experience; screen-based participants primarily inspect, annotate and communicate, rather than receiving identical authoring controls. [S6, S7, S8]

Native Quest versus tethered PC nuances

- **Standalone:** VR modeling, editing, passthrough and Collab Rooms work on supported Quest headsets, with limits such as one orthographic viewpoint at a time.
- **Exports:** Vendor documentation says offline/local FBX export is unavailable in standalone Quest; cloud or platform export workflows differ.
- **MR:** Passthrough is available on standalone supported headsets; depth occlusion has been in Labs for selected Quest 3-class headsets.
- **Collaboration:** Rooms can be joined by headset or screen; screen views include annotations, microphone, viewpoints and reviewer workflows.
- **Design lesson:** expose a single world to multiple participation roles, but do not force every device to have the same full feature set.

05 | Horizon Worlds: a scene and interaction editor, not a sculpting tool

Historical in-headset building UX

Earlier Horizon Worlds versions included spatial menus for shapes, assets, sounds and gizmos. Creators grabbed primitives from a panel, positioned them in the world, duplicated or recolored objects, grouped assemblies, and added interactive components. Typical building operations used hands/controllers for spatial selection and direct manipulation. The screenshot atlas identifies this as **historical VR build UI**, not a promise about 2026 Quest editing access.

Desktop Worlds Editor: evidenced interface and feature inventory

Region/tool	Function	Interaction model
Scene viewport	Navigate and place objects, inspect transforms, test world	Mouse/keyboard/gizmo and toolbar navigation
Hierarchy	Select parent/child entities, re-parent and organize content	Tree list, names, visibility and hierarchy operations
Asset Library	Search and insert prefabs, meshes, audio, textures and templates	Drag/click assets into scenes; import workflows
Properties/Inspector	Read and adjust entity transforms, behavior, rendering options	Numeric fields, toggles and property panels
Transform gizmos	Move, rotate, scale and snap objects	Axis handles plus grid and snap tools
TypeScript scripts	Handle entity events, state, collisions, logic and animations	IDE-linked code, attach component to entity
Runtime gizmos	Physics/grab, audio, particles, triggers and other interactive entity types	Configurable objects referenced from code
Testing/debugging	Preview world, inspect issues, script logs and performance	Play/test commands, console and diagnostics
Publishing/social	Publish worlds to supported targets, avatars, visits and multi-user events	World runtime and access controls
AI-assisted authoring	Creator Assistant / style references in 2025 releases	Optional workflow assistant; not core geometry editor

Horizon technical lesson

Worlds exposes a crucial separation: source assets, scene objects, object behavior, avatar/social systems and runtime visitors are distinct from each other. A Quest-native creation app should retain this separation rather than turning every scene object into a sculpt. An interactive door, for example, is a mesh plus collider, pivot, behavior script and permissions, not merely a shape.

2026 status and caution

In early 2026 Meta announced a mobile-first separation of Horizon Worlds from its Quest app ecosystem and initially described a full VR shutdown. Following creator/user objections, CTO Andrew Bosworth stated that **existing VR worlds would continue for the foreseeable future**. Subsequent coverage indicates future VR world publication remains constrained and the product emphasis is mobile. This is an evolving roadmap, not a foundation on which a new independent Quest application should rely. [S9, S10]

> **Architecture decision.** Use Meta XR or OpenXR capabilities and your own application/package, content storage, identity and permissions. Do not assume Horizon Worlds supports third-party plugins, importing its proprietary world scene representation, or embedding its editor into your new app.

06 | Multibrush: the central multiplayer precedent

Rendever built Multibrush after the 2021 release of Tilt Brush source code. It brought shared 3D painting to standalone Quest, and in 2022 documented avatar integration, passthrough, brush numbering, environmental controls and improved multiplayer synchronization. That combination matters: it demonstrates that a collaboratively editable creative scene does not need to be streamed from a desktop GPU. [S11, S12]

Feature/function inventory, distinguished by evidence

Feature	Observed/documented function	Engineering interpretation
3D painting strokes	Controller position/orientation generates visible spatial paint	Append-oriented stroke records
Brush palette	Select among visually styled brush materials	Preset-based mesh/shader types
Color selection	Change emitted stroke color	Material/color per stroke
Stroke width / paint parameters	Manipulate paint feel and appearance	Width/control samples
Room-sharing	Multiple artists in one canvas/room	Session/shared document
Avatar presence	Meta avatar representation in shared sessions	Presence separate from editing content
Mixed reality passthrough	Overlay paint in physical surroundings	Compositor passthrough camera
Environment panel	Select/change studio surroundings	Non-authoritative scene environment config
Private-room access	Room admission and identity checks	Membership and authorization
Undo/edit/select	Tilt Brush-derived editing workflows; multiplayer selection once had sync fixes	Per-object provenance and deterministic operation log
Saving/export	Tilt Brush lineage provides artwork file and geometry workflows; validate exact current app format	Versioned scene serialization and conversion
Shared tool parity	Developer explicitly notes that some tools are absent in shared canvases	Do not assume every solo editing command is network-safe

What the user actually does

A person selects a brush on a controller-attached palette, chooses color/scale, points to empty 3D space, holds the drawing trigger and moves their arm. Another person sees the evolving stroke and continues drawing in the same room. Both may walk around, change viewpoints, and work from different scales. The brush can be modeled as sampled positions, quaternions, pressure/width and brush-preset ID, then tessellated locally into line, ribbon or tube geometry.

> **The important limitation.** Original MultiBrush documentation says the full toolset exists in solo canvases but some tools are still being implemented in multiplayer rooms. That matters for undo, transform and stroke selection, which become synchronization problems rather than purely local UI actions. Avoid describing Multibrush as complete real-time multiplayer sculpting.

07 | A four-layer shared creation system

Layer	Data type / storage	Use case	Best inspiration
A. Spatial marks	Stroke sequences: poses, radius, color, preset, path samples	Draw lines, motion, hair, paint, light strokes	Multibrush + Gravity Sketch
B. Editable geometry	Curves, control points, surfaces, primitives, SubD cages	Build surfaces, furniture, forms, guides	Gravity Sketch
C. Local clay volumes	Sparse SDF tiles + ordered edit stamps + mip/LOD mesh	Refine an expressive sculpt, dent and smooth	Modeler
D. World objects	Mesh assets + transforms + colliders + components + scripts	Interactive rooms, doors, characters and lights	Horizon Worlds

Scene model: one hierarchy, distinct representations

```
Scene { id, schemaVersion, coordinateSystem, participants, layers, objects }
Object { id, type, parentId, transform, visible, ownerId, permissions, version }
Stroke { id, brushPreset, samples[], materialId, bounds, authorId }
Surface { id, curves[], topology, constraints, materialId, authorId }
Clay { id, primitiveTree?, sdfBrickRefs[], stampLog[], meshLODs[] }
Actor { id, meshRef, collider, behaviors[], state, interactionRules }
Layer { id, visibility, locked, editorRole, children[] }
```

Coordinate transforms must be explicit: creator-local tracking space, shared project space, object-local space, and optional colocated physical-anchor space are not interchangeable. All persisted geometry should be in project space with canonical units (meters) and stable object IDs. Rendering caches are disposable; parametric source data and authoring history are durable.

Choose export and exchange formats by purpose

Format	Use	Caveat
glTF / GLB	Portable runtime assets, meshes, materials, animations	Not sufficient to preserve editable clay or authoring history
USD / USDZ	Rich scene interchange and variants where pipeline supports them	Schema/tool version mismatches need tests
OBJ / FBX	Legacy mesh exchange with creation applications	Often lossy for materials, parametric structure and behaviors
Native project JSON + binary blobs	Authoritative editing semantics, revision history, permissions	Define schema and migration policy yourself
SDF brick volume package	Editable clay regions and multires meshes	Custom representation requiring native loader

Ownership is an object-level feature. Keep authorId, asset license, import origin, parent provenance, timestamps and agreed permissions as first-class properties. A creator should be able to export their own finished world or contribution, subject to permissions on third-party assets. Publishing must not silently transfer ownership.

08 | Multiplayer engineering: synchronize intentions, not rendered meshes

Four active creators, with viewers as a later separate load class

An initial target of four simultaneous editors is technically reasonable for a scoped product, but it must be proven in on-device profiling with representative scenes. Forty visitors can be considered later as a distinct low-authority runtime mode, not forty active sculptors. Room voice, tracking poses, editor operations, and persistent world snapshots require different channels and priorities.

Channel	Transport concept	Update discipline
Head/hands/avatars	Unreliable realtime packets, interpolation	Quantized poses at modest presence rates, priority by distance
In-progress brush motion	Unreliable short deltas while moving	Send small point batches, interpolate locally
Committed creative commands	Reliable ordered per-object/event stream	Server validates and persists before final commit
World snapshot and assets	Reliable chunked downloads, CDN/object store	Load by zone, background and incremental LOD
Voice	Optimized voice SDK with spatial mixing	Independent route with mute, block and privacy
Permissions + membership	Authenticated backend events	Server-authoritative role/lease changes

Per representation, use different concurrency rules

Edit type	Proposed conflict policy	What NOT to do
Append stroke	Unique stroke IDs; provisional streaming, immutable finalized samples	Broadcast a fully rebuilt triangle mesh per frame
Transform/group/rename	Short per-object edit lease; last acknowledged version	Allow two users to drag same object without arbitration
Curve/control points	Lease specific object or curve segment; server revisions	Silent last-writer-wins on topology edits
SubD topology	Exclusive topology lock then authoritative update	Pretend arbitrary concurrent edge edits commute
SDF clay	Partition into sparse bricks; order stamps within overlapping affected regions	Accept concurrent noncommutative sculpt stamps unsorted
Materials/environment	Property-level revisions, broadcast change	Rebuild every scene entity for color selection
Delete/undo	Tombstone or compensating operation with ownership and permissions	Undo another editor's unrelated operation

Minimal command protocol with authoritative sequencing

```
Client -> server:
  {opId, actorId, objectId, baseVersion, kind, payload, affectedBounds}
Server:
  validate identity + permissions + tool-specific bounds + version/lease
  assign monotonically increasing scene revision and object revision
  persist event; broadcast accepted operation + canonical revision
Client:
  locally preview unacknowledged drawing, reconcile authoritative replay
  checkpoint every N accepted operations; recover from snapshot + log
```

Why SDF stamping is the expensive exception

Modeler-style clay brushes may deposit SDF union/subtract, material and smoothing effects. Concurrent strokes are not automatically commutative: two users smoothing and subtracting the same area can get different outcomes based on order. Determine touched sparse bricks by brush AABB; use a per-brick revision/short lock or globally ordered operation list for overlapping regions; process meshing and LOD asynchronously. The editor should show a preview immediately, then reconcile when the accepted stamp completes.

09 | Quest performance architecture and budgets

Rendering profile

- **Baseline:** Test Quest 3 and Quest 3S on-device at 72 Hz or higher supported configurations; treat 13.9 ms as the full 72 Hz frame window, and leave thermal margin.
- **Strokes:** Batch brush meshes by shader/material; reuse mesh pools and GPU buffers. Far strokes can use simplified tubes or ribbons, fixed-width approximations and reduced shader complexity.
- **Surfaces:** Cache tessellation, avoid unnecessary rebuilds, use mesh LOD and static batching/instancing for stable geometry; throttle edits on large control cages.
- **Clay:** Sparse brick pages and per-region meshing; cap volume dimensions, active resolutions, stamp rate and brushes; mesh on worker jobs; use cross-frame work budgets.
- **Visuals:** Baked lighting, limited dynamic shadows, reduced transparent layer overdraw, avoid expensive post-processing; profile passthrough and other application layers separately.
- **Netcode:** Never let networking deserialization or large room snapshots block rendering. Decompress and instantiate progressively with versioned job queues.
- **Quality adaptations:** Fixed foveated rendering, dynamic resolution where tested, adaptive LOD, distant collaborator frustum filtering, GPU timing diagnostics and Android memory instrumentation.

Starter acceptance targets: internal goals, not published device guarantees

Metric	Prototype target	Measurement
Motion-to-render responsiveness	Maintain 72 FPS device requirement throughout basic sculpt/play	OVR Metrics / profiler under worst-case multiplayer load
Controller-to-local-preview	First plausible preview in same or next frame	Timestamp draw action to geometry preview visibility
Collaborator stroke display	Perceptually continuous despite typical network jitter	Remote visual traces; p50/p95; packet-loss simulation
Concurrent creators	4 connected editors drawing in representative scene	Stress tests with avatars, voice and edit operations
Undo safety	Never revert unrelated edits from other users	Automated replay/fuzz conflict suite
Reconnect	Restore exact committed scene and own pending operations	Disconnect/rejoin during strokes and asset fetch
Export correctness	No transforms lost; inspect in independent viewer	GLB import sanity; source schema round trips

Profile render time, CPU main/render worker times, GPU time, texture/mesh memory, garbage collection spikes, network jitter, draw calls, triangle count, active SDF bricks and asset-load latency. User comfort depends on stable output, not average FPS alone. Meta documents a 72-FPS minimum for interactive apps and provides Unity runtime quality controls. [S3, S13]

10 | Shared headset UX: keep the world, simplify the controls

Proposed role-based two-hand system

User intention	Proposed mapping	Source inspiration
Draw/sculpt with one hand	Dominant trigger, continuous preview; button quick-swaps paired tool	Modeler and Multibrush
Choose tool family	Non-dominant thumb button, hold for a compact floating palette	Gravity Sketch and Modeler
Adjust size/intensity	Dominant stick while tool active; haptic indication at thresholds	Modeler / Gravity Sketch
Move the whole world	Either grip moves; two grips scale; snap to room-scale/home	Both Modeler and Gravity Sketch
Select/edit object	Tap/select ray at distance or grab at reach; handles appear only when needed	Gravity Sketch / Horizon
Change precision	Explicit toggle between free placement, axis/surface snap, fine adjust	Modeler / Gravity Sketch
Find previous edit	Undo last personal committed command; inspect collaborative history	Multibrush / Modeler with multiplayer correction
Change scene mode	Wrist or palette tabs: Draw, Shape, Clay, World, Review	All four
Invite a friend	Room-code or deep link; immediate private room; readable roles	Gravity Sketch
Enter visit mode	Lock edit tools, show play/hear/tell or inspect affordances	Worlds creator vs visitor separation

Productivity, accessibility, and collaborative trust

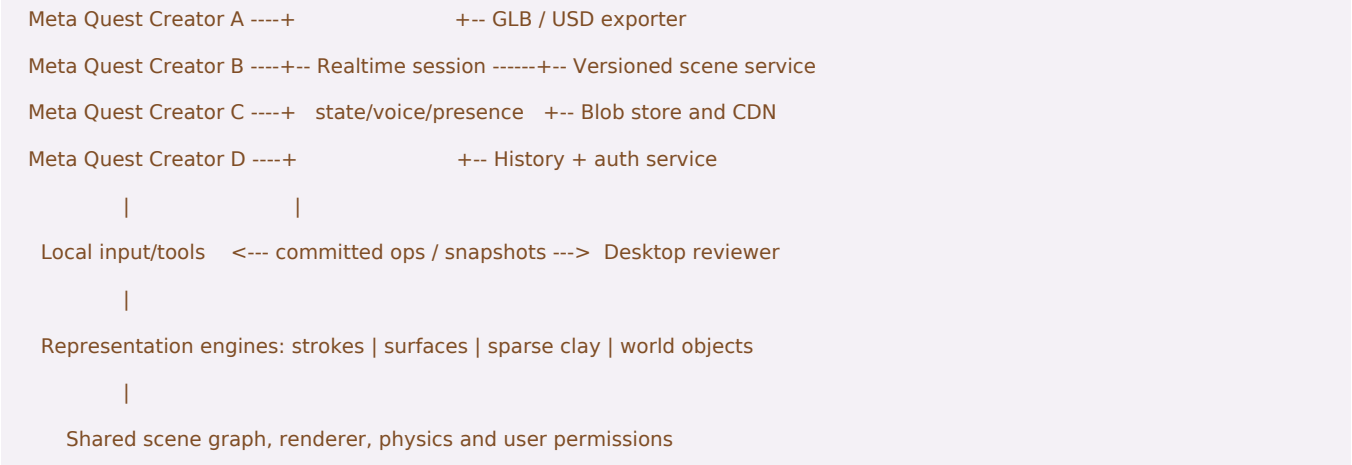
- **Seated mode:** Adjustable reach, controller cursor offset, easy scene recenter, no mandatory floor-bend or behind-body interaction.
- **Handedness:** Explicit mirror and rebind controls; do not bake right-hand assumptions into tutorials or avatar pose.
- **Fatigue:** Toggle-to-open alternative to hold-to-open menus; minimize sustained trigger squeeze; provide short-stroke assist and pressure curves.
- **Spatial clarity:** Keep fewer tools visible by default, show active brush state, and avoid menus that occlude the creation or another person.
- **Comfort:** Avoid forced locomotion, flashing effects, surprise camera movement, motion-coupled UI jumps, and high-frequency haptic noise.

- **Collaborative cues:** Show whose object is selected, who is holding a lock, and what will happen before deleting or replacing shared work.
- **Safety:** Private rooms by default; invite links, per-person mute/block/kick/report, content moderation and role-aware edit access.
- **Consent:** Voice capture, creator lore and visit traces off by default unless users explicitly choose to record or publish.

Why a compact five-tab UX is better than porting desktop folds

Mode	Default tools visible	Deeper tools when requested
DRAW	3D Ink, Tube, Ribbon, Eraser, Color, Width	Brush styles, smoothing, effects, mirror, replay
SHAPE	Primitive, Curve, Surface, Group, Transform	SubD topology, revolve, measurement, snap
CLAY	Add, Erase, Smooth, Warp, Resolution	SDF brick options, stamps, split/crop, paint
WORLD	Place prop, Light, Sound, Interact, Test	Behaviors, physics, animation, mesh assets
REVIEW	View, Point, Annotate, Invite, Save	Versions, credits, export, recording permissions

11 | Runtime architecture diagram and components



Suggested technology choices and their boundaries

Subsystem	Recommended starting option	Design caveat
Runtime	Unity LTS on Android/Quest + Meta XR SDK/OpenXR, URP	Pin package versions; regression-test each headset OS release
Network	Photon Fusion or comparable state/command replication service	Do not assume generic transforms solve collaborative authoring
Voice	Photon Voice or platform-supported spatial voice SDK	Consent, mute, cross-role routing, safety moderation
Data	Schema-versioned native scene JSON + chunked binary asset objects	Never rely only on emitted GLB for source editing
Strokes	Custom append-stream spline/ribbon/tube renderer; pool resources	Brush shading and overdraw matter on mobile
Geometry	Mesh/curve/primitive representation, editable control cages	Incremental tessellation and strict topology locks
Clay	Sparse chunked SDF with versioned stamps and async meshing	Optional post-MVP; costly to debug across devices
Scene logic	Sandboxed event-component set first; restricted scripts later	Do not let arbitrary user code execute in editor sessions
Build/storage	Addressable/content hash assets, local cache, server snapshots	Offline creator changes require explicit merging
MR colocation	Shared Spatial Anchors + group sessions where physical space is shared	Remote rooms do not need shared physical anchors

Meta publishes a Unity Shared Spatial Anchors sample with user or group alignment and Photon networking, and a Photon Fusion colocation sample that separates discovery, connection and spatial alignment. The app still needs its own collaboration policy and document synchronization system. [S14, S15]

12 | Failure modes and honest tradeoffs

Failure	Likely root	Mitigation
Frame spikes while clay stamping	SDF recompute and remesh on main thread	Sparse regions, worker jobs, capped active resolution
Other user's stroke jumps	Ordering/jitter or inconsistent origin	Local preview, canonical shared coordinates, ack/replay
Group edits collide	No object ownership/lease	Per-object short locks and explicit handoff UI
Undo destroys friend's work	Scene-global undo index	Author-scoped command history and compensating ops
Huge scene fails to load	Unbounded mesh and material memory	World streaming, LOD, content hashes and budgets
MR alignment drifts	Anchor tracking / large distance from anchor	Spatial anchor strategy and reset/recalibration
Room harassment or destructive edits	Uncontrolled permission and voice	Private access, mute/block, rate limits, rollback
Imported scene looks different	Materials/scale/schema mismatch	Validate conversion with named asset-level constraints
Creator loses authorship on publish	No provenance or export guarantees	Persist author/asset licenses; explicit export rights
Developer build depends on Horizons roadmap	External publishing platform governance	Ship as independent Quest-native application

A clear separation between proven and proposed

- **Proven by existing products:** Standalone spatial painting, VR sketching, multiuser sessions, avatar presence, cloud rooms, sculpting via PC and social world runtime.
- **Already supported by Quest tooling:** Unity-native hand/controller interactions, runtime performance controls, network samples, spatial anchors and local colocation.
- **Engineering proposal requiring prototyping:** A shared document that stores all four kinds of edits with consistent undo, author provenance, exports and material compatibility.
- **Highest risk:** Dense multiplayer SDF editing and cross-representation editing. These are not solved merely by buying a networking SDK.

13 | Build sequence and delivery gates

Phase	Build only this	Demonstrate before adding next phase
0 | Spike, 2-3 weeks	Quest scene, bimanual manipulation, remote stroke preview, 2 users	Shared 6DoF anchors and real device performance
1 | Social sketch, ~8-12 weeks	4 editors, stroke palettes, sync/replay, layers, permissions, save/reconnect	Continuous drawing, no lost strokes, safe creator undo
2 | Shape authoring	Curve edit, primitives, transforms, groups, snaps, import/export GLB	Round-trip geometry, topology locks, smooth large edits
3 | World scene	Asset library, lighting, sound, animation presets, runtime interactions	Creator can build and visit scene with friends
4 | Limited clay	Single-region sparse SDF tools, controlled resolutions, async meshes	Measured mobile thermal/frame budgets and exact replay
5 | Publisher	Exported worlds, provenance, viewer roles, desktop review, larger audiences	Permissions, performance, rollback, licensing acceptance

MVP definition. A four-person shared room where everyone can sketch, place primitives, shape a few editable surfaces, assemble a simple room, save it, revisit it, and export an owned GLB. Ship that before claiming full Substance-equivalent sculpting or forty-person world visitation. Timeline estimates assume a capable team and are planning ranges, not commitments.

Immediate engineering artifacts to create

- A 15-operation authoritative scene command schema and test fixture with replay/undo.
- A 12-action Quest controller interaction prototype with support/tool hand interchange.
- A performance benchmark room featuring 4 avatars, voice, 1,000 strokes, 100 editable objects and staged clay tiles.
- A scene export validator with author, licensing and attribution metadata.
- A usability test protocol: experienced VR creators, first-time Quest users, seated creators, people using left-hand layout, and co-creators editing simultaneously.
- A 4-to-40 user architecture note separating active collaborative editors from passive visitors and defining service and memory costs.

14 | Screenshot atlas: actual captured interfaces and controllers

The accompanying HTML screenshot atlas loads images directly from Adobe, Gravity Sketch, Meta documentation, Rendever and identified editorial sources. Below is the same provenance index with live links. Adobe and Gravity Sketch manufacturer controller images are identified as annotated diagrams; the other images are actual application screenshots unless otherwise stated. Because source image services block direct embedding into this PDF build environment, **the screenshots themselves are in the HTML atlas, not embedded pixels in this PDF**.

01. Adobe: desktop sculpting viewport - Adobe / Substance Modeler

Open actual image | Open source page

02. Adobe: desktop interface and Color Picker - Adobe / Substance Modeler

Open actual image | Open source page

03. Adobe: VR floating panels - Adobe / Substance Modeler

Open actual image | Open source page

04. Adobe: VR controller reference - Adobe / Substance Modeler

Open actual image | Open source page

05. Gravity Sketch: controllers with named buttons - Gravity Sketch official support

Open actual image | Open source page

06. Gravity Sketch: in-headset settings - Gravity Sketch official support

Open actual image | Open source page

07. Gravity Sketch: primitives menu - Creative Bloq editorial tutorial

Open actual image | Open source page

08. Horizon Worlds: historic VR build tool strip - Meta developer footage

Open actual image | Open source page

09. Horizon Worlds: asset library in VR - UploadVR screenshot

Open actual image | Open source page

10. Horizon Worlds: desktop editor - UploadVR / Vidyuu screenshot

Open actual image | Open source page

11. Horizon Worlds: imported custom model - MHCP Creator Made Docs

Open actual image | Open source page

12. Multibrush: in-world brushes and avatar - UploadVR screenshot

Open actual image | Open source page

13. Multibrush: passthrough promotion - Rendever official

Open actual image | Open source page

> **Screenshot timestamps matter.** Gravity Sketch interface images reflect the documentation revision; Multibrush examples are from 2022; Horizon Worlds VR build screenshots are from a historical product version and must not be interpreted as current in-headset authoring availability.

15 | Cited references and evidence trail

S1. Adobe technical requirements

https://helpx.adobe.com/substance-3d-modeler/desktop/technical-support/technical-requirements.html

S2. Adobe supported VR devices

https://helpx.adobe.com/substance-3d-modeler/desktop/getting-started/supported-vr-headsets.html

S3. Meta Quest Unity performance analysis

https://developers.meta.com/vr/documentation/unity/unity-perf/

S4. Adobe primitive and clay workflows

https://helpx.adobe.com/substance-3d-modeler/desktop/create-with-primitives/create-with-primitives.html

S5. Adobe VR shortcut tables

https://helpx.adobe.com/substance-3d-modeler/desktop/getting-started/controls/vr-shortcuts-and-controls.html

S6. Gravity Sketch creation tools

https://help.gravitysketch.com/creation-tools

S7. Gravity Sketch controller layout

https://help.gravitysketch.com/hc/en-us/articles/9380640515485-Controllers-Layout

S8. Gravity Sketch collaboration documentation

https://help.gravitysketch.com/collaboration

S9. Meta 2026 Worlds platform strategy

https://developers.meta.com/vr/blog/2026-vr-state-of-the-union-horizon-mobile-focus/

S10. March 2026 VR Worlds reversal reporting

https://www.uploadvr.com/meta-will-keep-horizon-worlds-vr-support-for-existing-games/

S11. Rendever original Multibrush launch

https://blog.rendever.com/2021/02/04/news-multibrush-by-rendever-now-available-on-sidequest-providing-artists-with-a-collaborative-tool-in-virtual-reality

S12. Rendever avatar and passthrough release

https://blog.rendever.com/multibrush-avatars-passthrough

S13. Meta Unity performance settings sample

https://developers.meta.com/vr/documentation/unity/unity-sample-performance-settings/

S14. Meta Unity shared spatial anchors sample

https://developers.meta.com/vr/documentation/unity/unity-sample-shared-spatial-anchors/

S15. Meta Photon Fusion colocation sample

https://developers.meta.com/vr/documentation/unity/unity-sample-colocation-fusion/

S16. Adobe SDF technical description

https://helpx.adobe.com/substance-3d-modeler/desktop/technical-support/how-does-modeler-work.html

S17. Adobe full tool inventory

https://helpx.adobe.com/substance-3d-modeler/desktop/create-with-clay/create-with-clay.html

S18. Gravity Sketch actual sketching tool categories

https://help.gravitysketch.com/hc/en-us/articles/11470059062429-sketching-tools

S19. Meta Worlds desktop editor and TypeScript

https://developers.meta.com/vr/blog/worlds/gdc-2025-horizon-worlds-create-earn-bonuses-desktop-editor-tools/

S20. Multibrush partial multiuser tool parity

https://www.vrwiki.cs.brown.edu/vr-modeling-software/multibrush

S21. Gravity Sketch Quest feature differences

https://help.gravitysketch.com/hc/en-us/articles/5902379957661-are-there-any-features-missing-on-the-standalone-quest-version-of-gravity-sketch

S22. Gravity Sketch current Labs features

https://help.gravitysketch.com/hc/en-us/articles/27153863233693-current-labs-features

S23. Meta scripting example

https://developers.meta.com/horizon-worlds/learn/videos/scripting-your-world/

S24. Open Brush open-source code and multiplayer hooks

https://github.com/icosa-foundation/open-brush

END OF ENGINEERING REFERENCE